

1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Sample Data
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

print("Raw Data")
print(df)

# Data Preprocessing
df["Employee_Name"] = df["Employee_Name"].str.strip()

median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

df["Join_Date"] = pd.to_datetime(df["Join_Date"])

df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("\nPreprocessed Data")
print(df)

# Aggregation
dept_summary = (
    df.groupby("Department")
    .agg(
        Avg_Salary=("Salary", "mean"),
```

```
Total_Employees=("Employee_Name", "count")
)
.reset_index()
)

print("\nDepartment Summary")
print(dept_summary)

# Visualization
sns.set_theme(style="whitegrid")

fig, axes = plt.subplots(1,2, figsize=(14,5))

sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    palette="Blues_d",
    legend=False,
    ax=axes[0]
)

axes[0].set_title("Average Salary by Department")

sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1]
)

axes[1].set_title("Salary vs Years of Service")

plt.tight_layout()
plt.show()
```

Raw Data

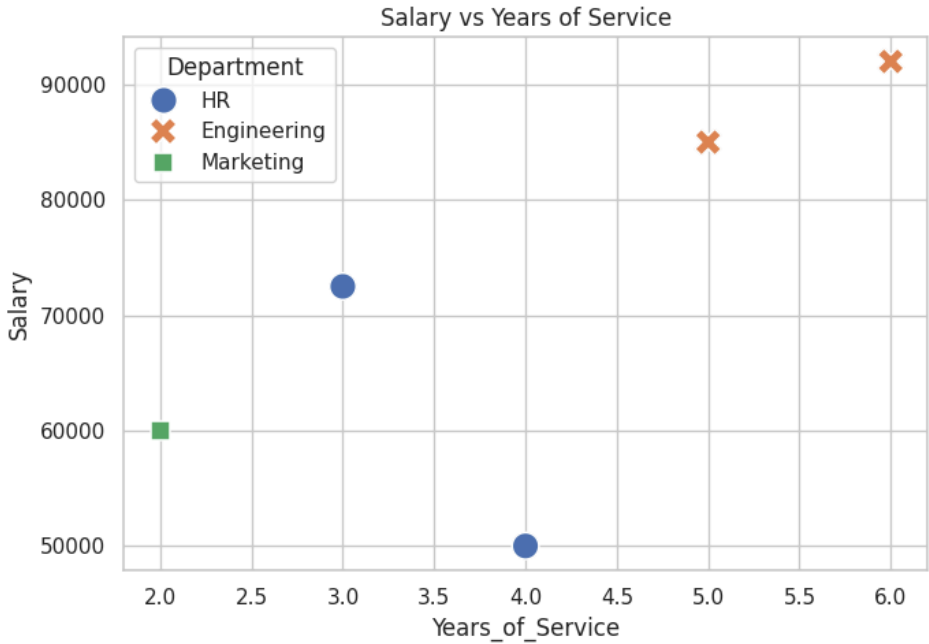
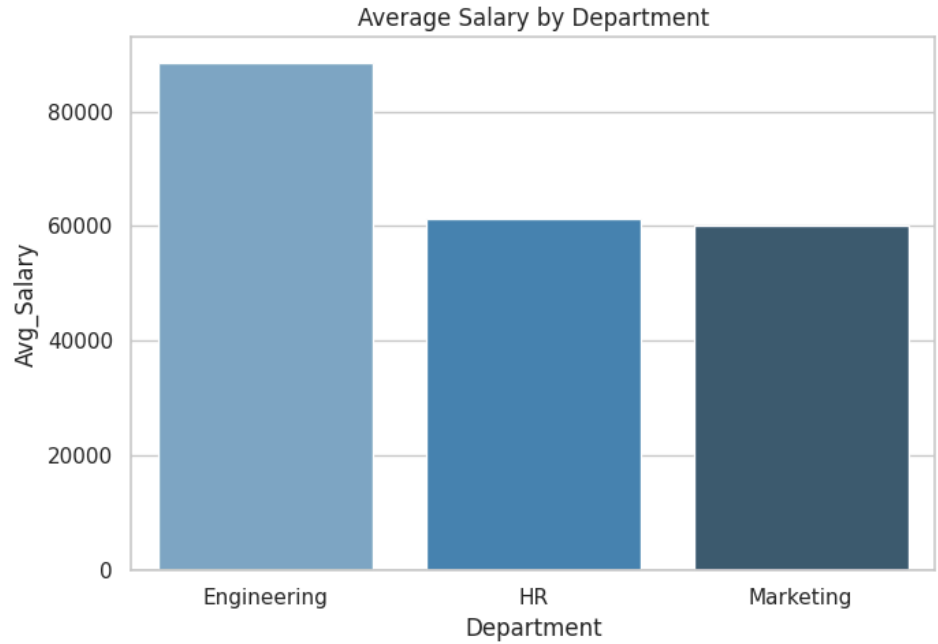
	Employee_Name	Department	Salary	Join_Date
0	Alice	HR	50000.0	2022-01-15
1	Bob	Engineering	85000.0	2021-06-20
2	Charlie	HR	NaN	2023-03-11
3	Diana	Engineering	92000.0	2020-11-01
4	Evan	Marketing	60000.0	2024-02-28

Preprocessed Data

	Employee_Name	Department	Salary	Join_Date	Years_of_Service
0	Alice	HR	50000.0	2022-01-15	4
1	Bob	Engineering	85000.0	2021-06-20	5
2	Charlie	HR	72500.0	2023-03-11	3
3	Diana	Engineering	92000.0	2020-11-01	6
4	Evan	Marketing	60000.0	2024-02-28	2

Department Summary

	Department	Avg_Salary	Total_Employees
0	Engineering	88500.0	2
1	HR	61250.0	2
2	Marketing	60000.0	1



```
import pandas as pd

df = pd.DataFrame()

df["Name"] = ["John", "Emma", "Liam", "Olivia"]
df["Age"] = [20, 19, 21, 18]
df["Student"] = [True, True, False, True]

print(df)
```

	Name	Age	Student
0	John	20	True
1	Emma	19	True
2	Liam	21	False
3	Olivia	18	True

3

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["John", "Emma", "Liam", "Olivia"],
    "Age": [20, 19, 21, 18],
    "Student": [True, True, False, True]
})

print("Original DataFrame")
print(df)

new_row = pd.DataFrame(
    [
        ["Sophia", 22, False],
    ],
    columns=["Name", "Age", "Student"]
)

df = pd.concat([df, new_row], ignore_index=True)

print("\nAfter Adding New Row")
print(df)
```

Original DataFrame

	Name	Age	Student
0	John	20	True
1	Emma	19	True
2	Liam	21	False
3	Olivia	18	True

```
After Adding New Row
   Name  Age  Student
0   John   20     True
1   Emma   19     True
2   Liam   21    False
3  Olivia   18     True
4  Sophia   22    False
```

4

```
import numpy as np
from scipy import stats

data = [10, 20, 30, 40, 50]

mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data)

print("Mean =", mean)
print("Median =", median)
print("Mode =", mode)
```

```
Mean = 30.0
Median = 30.0
Mode = ModeResult(mode=np.int64(10), count=np.int64(1))
```

5

```
from scipy.stats import norm

# Mean = 70
# Standard Deviation = 10
# Find probability for value 85

probability = norm.cdf(85, loc=70, scale=10)

print("Probability =", probability)
```

```
Probability = 0.9331927987311419
```

6

```
from scipy import stats

# Sample Data
data = [22, 25, 19, 24, 28, 30]

# Perform One Sample T-Test
t_stat, p_value = stats.ttest_1samp(data, 25)

print("T-Statistic =", t_stat)
print("P-Value =", p_value)

# Interpretation
if p_value < 0.05:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

```
T-Statistic = -0.2049800154226962
P-Value = 0.8456711339658732
Fail to Reject the Null Hypothesis
```

7

```
from scipy.stats import pearsonr

# Sample Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Pearson Correlation
corr, p_value = pearsonr(x, y)

print("Correlation =", corr)
print("P-Value =", p_value)
```

```
Correlation = 1.0
P-Value = 0.0
```

8

```
from scipy import linalg

# Coefficient Matrix
A = [
```

```
[3, 2],  
[1, 2]  
]  
  
# Constant Matrix  
B = [5, 5]  
  
# Solve Linear Equations  
solution = linalg.solve(A, B)  
  
print("Solution:")  
print(solution)
```

```
Solution:  
[0.  2.5]
```

9

```
from scipy.optimize import minimize  
  
# Objective Function  
def objective(x):  
    return x**2 + 4  
  
# Initial Guess  
result = minimize(objective, x0=5)  
  
print("Optimal Value of x:")  
print(result.x)  
  
print("\nMinimum Function Value:")  
print(result.fun)
```

```
Optimal Value of x:  
[-2.62955131e-08]
```

```
Minimum Function Value:  
4.000000000000001
```

10

```
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load Iris Dataset
iris = datasets.load_iris()

X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Model
model = LogisticRegression()

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", accuracy)

print("\nClassification Report")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

Accuracy = 1.0
```

```

Classification Report
              precision    recall  f1-score   support

         0       1.00      1.00      1.00        19
         1       1.00      1.00      1.00        13
         2       1.00      1.00      1.00        13

 accuracy          1.00
macro avg          1.00      1.00      1.00        45
weighted avg          1.00      1.00      1.00        45

```

```

Confusion Matrix
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

```

11

```

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load Dataset
iris = load_iris()

X = iris.data

# Create KMeans Model
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

# Train Model
kmeans.fit(X)

# Cluster Labels
labels = kmeans.labels_

print("Cluster Labels:")
print(labels)

# Cluster Centers
print("\nCluster Centers:")
print(kmeans.cluster_centers_)

```

```
# Visualization
plt.figure(figsize=(8,6))

plt.scatter(
    X[:,0],
    X[:,1],
    c=labels,
    cmap="viridis",
    s=60
)

plt.scatter(
    kmeans.cluster_centers_[:,0],
    kmeans.cluster_centers_[:,1],
    color="red",
    marker="X",
    s=200,
    label="Centroids"
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("K-Means Clustering on Iris Dataset")
plt.legend()

plt.show()
```

[illegible]

```
Cluster Centers:
[[5.9016129  2.7483871  4.39354839  1.43387097]
 [5.006      3.428      1.462      0.246      ]
 [6.85      3.07368421  5.74210526  2.07105263]]
```

